

Contents

Instructor Notes - TestAbility Workshop	1
How can we investigate Testability?	1
What does it mean to investigate Testability?	1
Mob Exercise: What presuppositions exist?	1
Technology Audit	2
Application Test Ability Audit	3

Instructor Notes - TestAbility Workshop

How can we investigate Testability?

- Look at system
- Try to test it
- From experience see what supports testing and what doesn't
- See what is hard and what is easy
- See what I can manipulate and what I can't
- What technology levels can I test at?
- What techniques can I apply?
- What tools can I use?
- What are the limits of my knowledge?

What does it mean to investigate Testability?

Specifically:

Can I test this? How well can I test this? What supports me in testing this?
What hinders me in testing this? What are the defaults associated with testing
this? What are the issues with testing this? Where the risks with testing this?

Generically:

- Technology
- Type of App
- Domain
- Do I understand functionality

Me in relation to the application

Mob Exercise: What presuppositions exist?

- Testability is important
- Testability is something we can investigate
- There is more to Testability than we might at first assume
- Investigating Testability can change how I test and is worth investigating

Technology Audit

- Java
 - JRE version? used on 1.9
 - Cross platform? should be - haven't tried
- CLI interface
 - Mobile? Don't know
 - Tool: Terminal used? Shell used?

Source code is available

- <https://github.com/eviltester/zombie-cli-text-adventure>
- <https://github.com/eviltester/restmud-cli-game-engine>
- Java Technology
 - Need to have a JRE
 - JRE might impact app depending on version
 - Should make it cross platform
 - Do we understand Java?
 - Do we need to understand Java?
 - How would understanding Java change the applications Testability?
- CLI interface
 - this might impact portability
- Application Architecture (3 logical tiers)
 - CLI Game Engine
 - Adventure Game Engine
 - Game Rules and Data

Do our tools count as 'technology' or are they for automatability?

- Java can be decompiled if source isn't available. Decompilers exist and are built into IntelliJ

This application is actually dependent on other applications, and the full game engine is used in the web version and has a lot of tests associated with it.

This game has 2 engines: CLI, Game Engine, and Game Rules - what am I testing? The rules, the CLI game engine? The Main Game Engine?

Is this the easiest way to test each of these?

Insights:

- If I don't know the technology I may not know what tools to use
- If I don't understand the programming language I cannot use the source as an oracle or way of learning

- I can't test below the level I understand e.g. I can't look at HTTP traffic if I don't understand it, or know that it exists

Application Test Ability Audit

Walkthrough test might reveal information

<https://github.com/eviltester/tech-web-test-restmud-game>

in \docs

There is also a map there

The src has the game definitions

<https://github.com/eviltester/tech-web-test-restmud-game/>

- `src/main/java/uk/co/compediumdev/restmud/gamedata`
- `/games/gamedefinitions/TechWebTest101GameDefinitionGenerator.java`
- CLI Args to 'unlock features for testing'
 - `-h` for help
 - `-v` for version
 - `-verbose`
 - `-cols 40` - reformat output
 - `-txtlog logfile.txt`
 - `-jsonlog jsonlog.txt`
 - `-debug`
 - `-nobackups`
 - `-commandslog`